BACKEND VALIDATION GUIDE

WHY DO WE NEED BACKEND VALIDATION?

Frontend validation is useful for giving users instant feedback, but it should **NOT** be treated as a security boundary.

Anything coming from the client should be treated as untrusted. A user can easily bypass frontend validation by:

- Sending requests through Postman
- Using curl
- Writing a custom script
- Modifying requests through a proxy
- Calling the API directly

That's why validation must also happen on the backend before the application processes the data.

1. TYPE VALIDATION

Make sure the received value has the expected data type.

- age should be a number
- email should be a string
- isAdmin should be a boolean

2. REQUIRED FIELD VALIDATION

Check that required fields are actually present in the request body.

Example: name, email, password

Don't assume the frontend will always send them.

3. FORMAT VALIDATION

Check whether data follows the expected pattern or standard structure.

- Email format
- Phone number format
- Date format
- UUID format
- URL format

4. LENGTH VALIDATION

Set reasonable minimum and maximum lengths to prevent unnecessarily large or malformed input.

Example (Username):

- Minimum: 3 characters
- Maximum: 30 characters

5. RANGE VALIDATION

Make sure numerical values fall within acceptable operational limits.

Examples:

- age >= 18
- quantity >= 1
- percentage >= 0 && percentage <= 100

6. ALLOWLIST VALIDATION

When possible, define what values **ARE** allowed instead of only trying to block bad values.

Example: role can only be "user" or "admin", instead of accepting any arbitrary string.

7. BUSINESS LOGIC VALIDATION

This is extremely important. Some data can be correctly formatted but still be invalid according

to your application's rules.

Scenario	Check / Rule
Product order	A product may exist, but it might be out of stock.
Discount application	A coupon may exist, but it might have expired.
Action authorization	A user may be logged in, but they may not have permission to perform an action.

These rules must be enforced on the backend.

8. UNKNOWN / EXTRA FIELDS

Don't blindly trust everything sent by the client. For important APIs, define the fields the endpoint actually accepts and reject or safely ignore unexpected fields according to your API design.

9. FILE UPLOAD VALIDATION

If users can upload files, validate attributes such as:

- File type
- File size
- File name
- File contents where appropriate

Never assume a file is safe just because its extension says .jpg or .pdf.

10. VALIDATION ≠ COMPLETE SECURITY

Backend validation is extremely important, but validation alone doesn't prevent every security

problem. You may also need:

- Authentication
- Authorization
- Output encoding
- Parameterized database queries
- Rate limiting
- CSRF protection where applicable
- Secure file handling
- Proper error handling

THE GOLDEN RULE

Validation Type	Primary Purpose
Frontend Validation	Better user experience
Backend Validation	Security + data integrity

NEVER TRUST CLIENT-SIDE VALIDATION.

Even if the frontend validates the data, the backend should validate it again before processing it.

EASY WAY TO REMEMBER

```
Anything coming from the client
             ↓
         UNTRUSTED
             ↓
  Backend validates it
             ↓
Only then process the data
```

Final Takeaway

- **Frontend validation tells the user:** *"Hey, this data looks wrong."*

- **Backend validation tells the application:** *"I won't process this data until it passes my rules."*